

南开大学

汇编语言与逆向技术课程实验报告

Lab9: REExercises (advanced)



学院：密码与网络空间安全学院

专业：信息安全

学号：2412266

姓名：杨滢

班级：信息安全

目录

1 实验时间	2
2 实验目的	2
3 实验环境	2
4 反汇编代码 Binary Ninja 视窗	2
5 task3 逆向分析	3
5.1 程序流程图	3
5.2 详细过程	3
5.2.1 阶段 1: 初始化与提示信息解密	3
5.2.2 阶段 2: 用户输入与长度检查	4
5.2.3 阶段 3: 硬编码验证数组	5
5.2.4 阶段 4: 输入验证逻辑	5
5.2.5 阶段 5: 失败处理	6
5.3 代码求解	6
5.4 数据结构	7
6 task4 逆向分析	7
6.1 程序流程图	7
6.2 详细过程	7
6.2.1 阶段 1: 初始化与输入读取	7
6.2.2 阶段 2: 计算输入长度	8
6.2.3 阶段 3: 长度验证	8
6.2.4 阶段 4: 字符串变换算法	8
6.2.5 阶段 5: 硬编码比较数据	9
6.2.6 阶段 6: 内存比较逻辑	9
6.2.7 解密算法推导	10
6.2.8 确输入字符串的计算公式	10
6.3 代码求解	10
6.4 数据结构	11
7 实验结果	11

1 实验时间

第 16 周星期一 9-10 节

2 实验目的

1. 进一步熟悉静态反汇编工具 Binary Ninja;
2. 熟悉将反汇编代码进行反编译的过程;
3. 掌握对于反编译伪代码的逆向分析;
4. 运用熟悉的编程语言, 实现简单的脚本编写

3 实验环境

Binary Ninja、PyCharm

4 反汇编代码 Binary Ninja 视窗

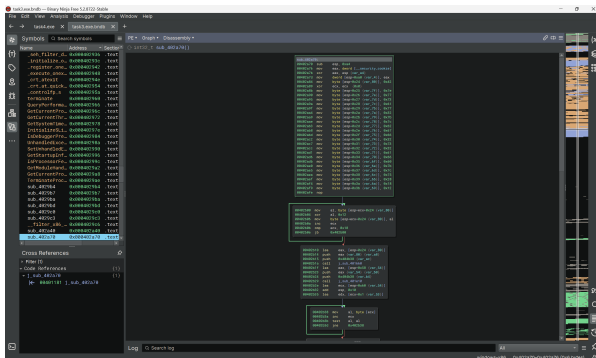


图 4.1: task3 反汇编代码图形化视窗

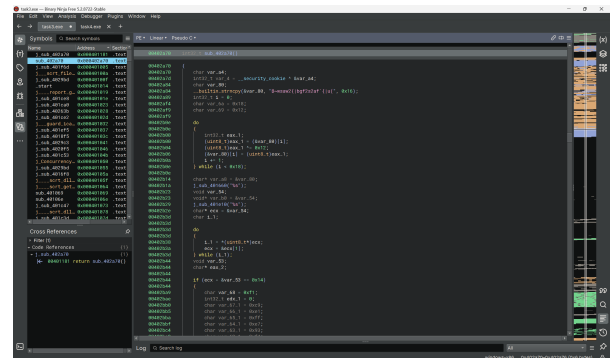


图 4.2: task3 反编译伪 C 代码

图 4.3: task3

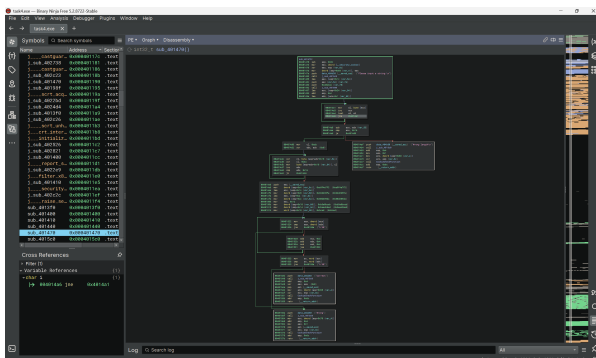


图 4.4: task4 反汇编代码图形化视窗

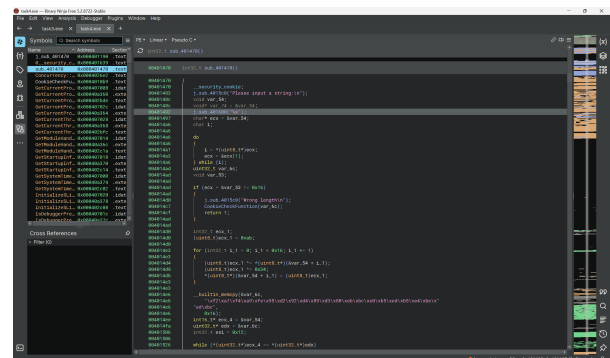
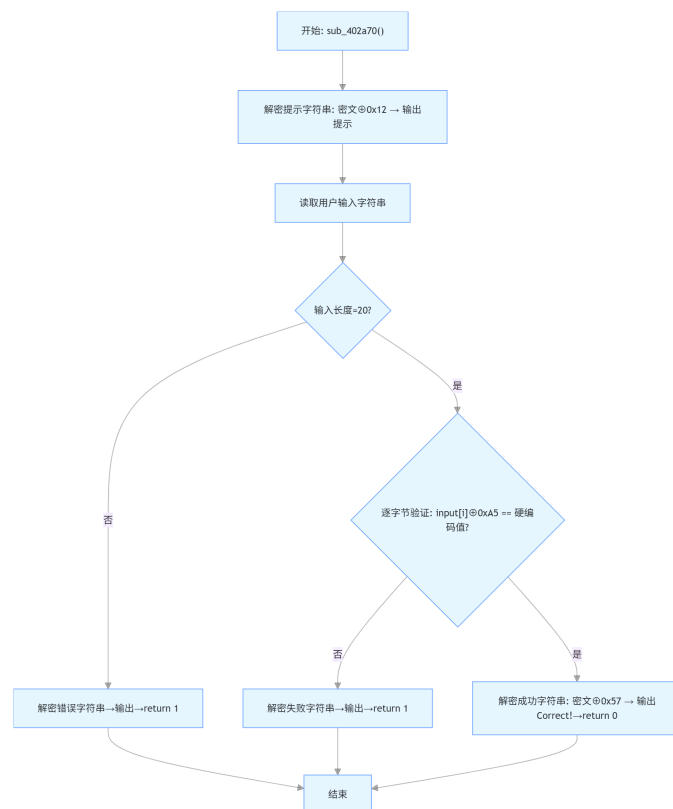


图 4.5: task4 反编译伪 C 代码

图 4.6: task4

5 task3 逆向分析

5.1 程序流程图



5.2 详细过程

5.2.1 阶段 1: 初始化与提示信息解密

解密计算:

- 原始字符串: "Bŵsaw2{|bgf2s2af{|u("
- 密钥: 0x12 (18)
- 解密公式: $cipher[i] \oplus 0x12 = plain[i]$
- 解密结果: "Try to reverse the first part!"

计算公式推导:

$$\text{设 } C = [c_0, c_1, \dots, c_{21}] = \text{"Bŵsaw2{|bgf2s2af{|u("}$$

$$K = 0x12$$

$$\text{则 } P = [p_0, p_1, \dots, p_{21}] = C \oplus K$$

$$p_i = c_i \oplus 0x12 \quad \text{for } i = 0, 1, \dots, 21$$

```
1 00402a70 int32_t sub_402a70() {
2 00402a70     char var_a4;
3 00402a7d     int32_t var_4 = __security_cookie ^ &var_a4;
4     // Windows栈保护机制，用于检测栈溢出攻击
5 00402a84     char var_80;
6 00402a84     __builtin_strncpy(&var_80, "B~wsaw2{|bgf2s2af`{|u(", 22);
7     // 将22字节的加密字符串复制到栈变量var_80中
8 00402a89     int32_t i = 0;
9 00402af4     char var_6a = 0x18;    // 后续会被修改
10 00402af9     char var_69 = 0x12;  // 后续会被修改
11 00402b0e     do {
12 00402b00         int32_t eax_1;
13 00402b00         (uint8_t)eax_1 = (&var_80)[i];
14 00402b04         (uint8_t)eax_1 ^= 18; // 异或操作：每个字节与0x12(18)异或
15 00402b06         (&var_80)[i] = (uint8_t)eax_1;
16 00402b0a         i += 1;
17 00402b0e     } while (i < 24);
18     // 循环解密：处理var_80开始的24个字节
19     // 注意：原始字符串只有22字节，但会继续处理var_6a和var_69
20 00402b14     char* var_a8 = &var_80;
21 00402b1a     j_sub_401660("%s");
22     // 打印解密后的提示信息
23 }
```

5.2.2 阶段 2：用户输入与长度检查

长度计算说明：设字符串长度为 n ，则：

- 起始地址 = $\&\text{var_54}$
- 结束地址 = $\&\text{var_54} + n + 1$ （指向'\0'后一个字节）
- 长度检查 = $(\&\text{var_54} + n + 1) - (\&\text{var_54} + 1) = n = 20$

```
1 00402b2e     char* ecx = &var_54;    // 指向输入字符串起始
2 00402b3d     char i_1;
3 00402b3d     do {
4 00402b38         i_1 = *(uint8_t*)ecx; // 读取当前字符
5 00402b3a         ecx = &ecx[1];      // 指针移动到下一个字符
6 00402b3d     } while (i_1);           // 循环直到遇到'\0'
7     // 遍历字符串计算长度
8 00402b44     void var_53;              // var_54的下一个字节
9 00402b44     char* eax_2;
```

```

10 00402b44     if (ecx - &var_53 == 20) {
11     // ecx - (&var_54 + 1) = 字符串长度
12     // 要求输入长度为20个字符
13 }

```

5.2.3 阶段 3：硬编码验证数组

验证数组的完整表示：

$$enc = \begin{bmatrix} 0xF1 & 0xC9 & 0xE1 & 0xFF & 0xE7 & 0x93 & 0xF4 & 0xEF & 0xD4 & 0xE8 \\ 0xEF & 0xC0 & 0xCE & 0xFC & 0xE2 & 0xD1 & 0xFD & 0xC0 & 0xF1 & 0xFC \end{bmatrix}$$

5.2.4 阶段 4：输入验证逻辑

验证公式推导：程序检查的条件为：

$$\text{input}[i] \oplus 0xA5 = \text{enc}[i] \quad \text{for } i = 0, 1, \dots, 19$$

正确输入的计算公式：

$$\text{input}[i] = \text{enc}[i] \oplus 0xA5$$

其中：

$$0xA5 = 165_{10} = 10100101_2$$

计算结果：

- 十六进制结果: 0x54, 0x6c, 0x44, 0x5a, 0x42, 0x36, 0x51, 0x4a, 0x71, 0x4d, 0x4a, 0x65, 0x6b, 0x59, 0x47, 0x74, 0x58, 0x65, 0x54, 0x59
- ASCII 结果: T1DZB6QJqMJekYGtXeTY
- 长度: 20 个字符

```

1 00402c22     while (((int32_t)*(uint8_t*)&var_54 + edx_1) ^ 165)
2 00402c22         == (uint32_t)&var_68[edx_1]) {
3     // 验证条件: input[edx_1] ^ 0xA5 == enc[edx_1]
4     // 等价于: input[edx_1] == enc[edx_1] ^ 0xA5
5 00402c22         edx_1 += 1;           // 移动到下一个字符
6 00402c28         if (edx_1 >= 20) { // 如果所有20个字符都验证通过
7 00402c2a             char var_9c = 20;
8 00402c2f             int32_t i_2 = 0;
9 00402c31             char var_9b;
10 00402c31             __builtin_strncpy(&var_9b, "8%%24#vW", 8);
11 00402c6e             do {
12 00402c60                 (uint8_t)eax_3 = (&var_9c)[i_2];
13 00402c64                 (uint8_t)eax_3 ^= 87; // 0x57

```

```
14 00402c66          (&var_9c)[i_2] = (uint8_t)eax_3;
15 00402c6a          i_2 += 1;
16 00402c6e          } while (i_2 < 9);
17 // 解密成功信息: "8%%24#vW" 0x57 = "Correct!"
18 00402c74          char* var_a8_1 = &var_9c;
19 00402c7a          j_sub_401660("%s\n");
20 // 打印"Correct!"
21 00402c8d          CookieCheckFunction(var_a4);
22 00402c98          return 0;          // 返回成功
23      }
24  }
25 }
```

5.2.5 阶段 5：失败处理

当用户输入不满足验证条件时，程序进入失败处理分支。这部分代码处理两种不同的失败情况：

1. 内容验证失败：输入长度正确（20 字节），但内容验证不通过
2. 长度验证失败：输入长度不等于 20 字节

5.3 代码求解

依据上述分析，可以编写 python 代码，运行得到结果。

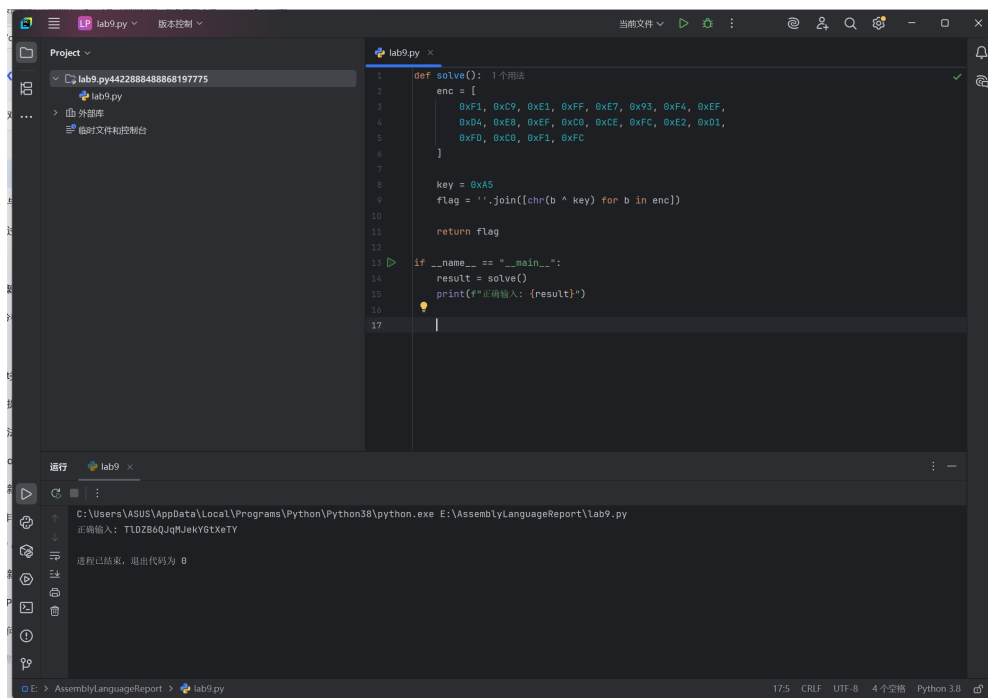


图 5.7: task3 python 代码运行截图

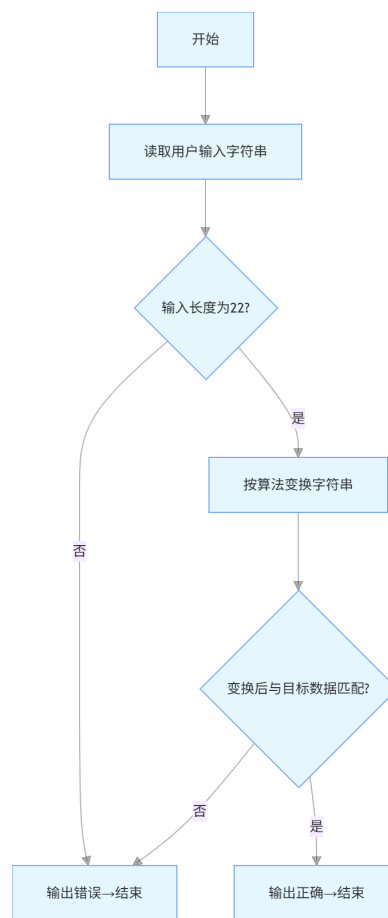
结果为：TlDZB6QJqMJekYGtXeTY

5.4 数据结构

核心的数据结构是基于栈的字符数组（字节数组），所有数据都是通过栈上的连续内存块来组织的。

6 task4 逆向分析

6.1 程序流程图



6.2 详细过程

6.2.1 阶段 1：初始化与输入读取

```
1 00401470  int32_t sub_401470() {
2 00401470      __security_cookie;    // 栈保护机制初始化
3 00401483      j_sub_4015c0("Please input a string:\n");// 输出提示信息"Please input
    a string:\n"
4 0040148c      void var_54;          // 定义输入缓冲区，大小为至少23字节
5 0040148c      void* var_74 = &var_54; // 保存缓冲区地址到var_74
```



```
6 00401492      j_sub_401600("%s");    // 读取用户输入
```

6.2.2 阶段 2：计算输入长度

长度计算原理：设输入字符串长度为 n ，则内存布局如下：

起始地址: $\&var_54$

字符串内容: $input[0], input[1], \dots, input[n-1]$

结束符: $'\0'$ (ASCII 0)

循环结束 ecx 位置: $\&var_54 + n + 1$

```
1 004014a6      do {                                // 开始遍历字符串
2 004014a1          i = *(uint8_t*)ecx; // 读取ecx指向的字符
3 004014a3          ecx = &ecx[1];    // ecx指针向后移动一个字节
4 004014a6      } while (i);                // 循环直到遇到字符'\0' (ASCII 0)
```

6.2.3 阶段 3：长度验证

长度要求：输入必须是 22 个字符（不包括结尾的 $\backslash 0$ ）。

```
1 004014ad      void var_53;                        // var_54的下一个字节地址
2 004014ad      if (ecx - &var_53 != 0x16) {
3 // 检查字符串长度n是否等于0x16(22)
4 004014d0          j_sub_4015c0("Wrong length\n"); // 如果长度不等于22，输出错误信息
5 004014c7          CookieCheckFunction(var_6c);
6 004014cf          return 1;                      // 返回1表示失败
7 }
```

6.2.4 阶段 4：字符串变换算法

算法形式化描述：

设: $state_0 = 0xAB$ （初始状态）

$input_i$ 为第 i 个原始输入字符

$transformed_i$ 为变换后的第 i 个字符

对于 $i = 0, 1, \dots, 21$:

$$state_{i+1} = state_i \oplus input_i \oplus 0x34$$

$$transformed_i = state_{i+1}$$

算法特点：

- 这是一个流密码算法，每个字符的加密依赖于前一个状态
- 初始状态固定为 0xAB
- 异或常数固定为 0x34
- 算法可逆，可以解密

```

1 004014d0      int32_t ecx_1;
2 004014d0      (uint8_t)ecx_1 = 0xab; // 初始化状态变量为0xAB
3 004014e3      for (int32_t i_1 = 0; i_1 < 0x16; i_1 += 1) {
4          // 循环22次(0x16)，对每个输入字符进行处理
5 004014d4          (uint8_t)ecx_1 ^= *(uint8_t*)&var_54 + i_1);
6          // state = state XOR input[i], 将当前状态与输入字符异
7 004014d8          (uint8_t)ecx_1 ^= 0x34;
8          // state = state XOR 0x34, 再与常数0x34异或
9 004014db          *(uint8_t*)&var_54 + i_1) = (uint8_t)ecx_1;
10 004014e3      }

```

6.2.5 阶段 5：硬编码比较数据

硬编码数组：

$$\text{expected} = \begin{bmatrix} 0xF2 & 0xA7 & 0xF4 & 0xA9 & 0xFE & 0x95 & 0xD2 & 0x92 \\ 0xD4 & 0x89 & 0xD3 & 0x80 & 0xEB & 0xBC & 0xE0 & 0xB5 \\ 0xED & 0xB5 & 0xE4 & 0xBE & 0xED & 0xBC & & \end{bmatrix}$$

```

1 004014e6      __builtin_memcpy(&var_6c,
2          "\xf2\xa7\xf4\xa9\xfe\x95\xd2\x92\xd4\x89\xd3\x80\xeb\xbc"
3          "\xe0\xb5\xed\xb5\xe4\xbe\xed\xbc",
4          0x16); // 复制22字节(0x16)的硬编码数据到var_6c

```

6.2.6 阶段 6：内存比较逻辑

比较策略优化：

- 使用 4 字节 (uint32_t) 为单位比较，提高效率
- 22 字节 = 5×4 字节 + 2 字节
- 先比较前 20 字节（5 次 4 字节比较）
- 最后比较剩余 2 字节

```

1 004014ee      int16_t* ecx_4 = &var_54; // 指向变换后的输入
2 004014fa      uint32_t* edx = &var_6c; // 指向硬编码数据

```

```

3 00401506      int32_t esi = 0x12;      // 剩余字节数: 18(0x12)
4 // 4字节单独处理
5 00401526      while (*(uint32_t*)ecx_4 == *(uint32_t*)edx) {
6 00401526          ecx_4 = &ecx_4[2];    // int16_t指针, +2表示+4字节
7 0040152b          edx = &edx[1];      // uint32_t指针, +1表示+4字节
8 0040152e          esi -= 4;            // 减少4字节
9 00401531          if (esi < 4) {        // 如果剩余字节少于4
10 00401533              int32_t eax_3;
11 00401533              (uint16_t)eax_3 = *(uint16_t*)ecx_4; // 比较最后2字节(16位)
12 00401539              if ((uint16_t)eax_3 != *(uint16_t*)edx)
13 00401540                  break;        // 不相等则跳出
14 00401540              j_sub_4015c0("Correct"); // 全部匹配, 输出"Correct"
15 00401551              CookieCheckFunction(var_6c);
16 00401559              return 0;        // 成功返回0
17          }
18      }

```

6.2.7 解密算法推导

已知加密算法:

$$\begin{cases} \text{state}_0 = 0xAB \\ \text{对于 } i = 0, 1, \dots, 21 : \\ \quad \text{state}_{i+1} = \text{state}_i \oplus \text{input}_i \oplus 0x34 \\ \quad \text{transformed}_i = \text{state}_{i+1} \end{cases}$$

解密算法 (已知 transformed_i , 求 input_i):

$$\text{input}_i = \text{state}_i \oplus \text{transformed}_i \oplus 0x34$$

其中 state_i 是前一个变换值 (对于 $i > 0$), $\text{state}_0 = 0xAB$ 。

6.2.8 确输入字符串的计算公式

将上述计算过程总结为一般公式:

$$\text{input}[i] = \begin{cases} 0xAB \oplus \text{expected}[0] \oplus 0x34 & \text{if } i = 0 \\ \text{expected}[i-1] \oplus \text{expected}[i] \oplus 0x34 & \text{if } 1 \leq i \leq 21 \end{cases}$$

6.3 代码求解

编写 python 代码, 运行得到结果。

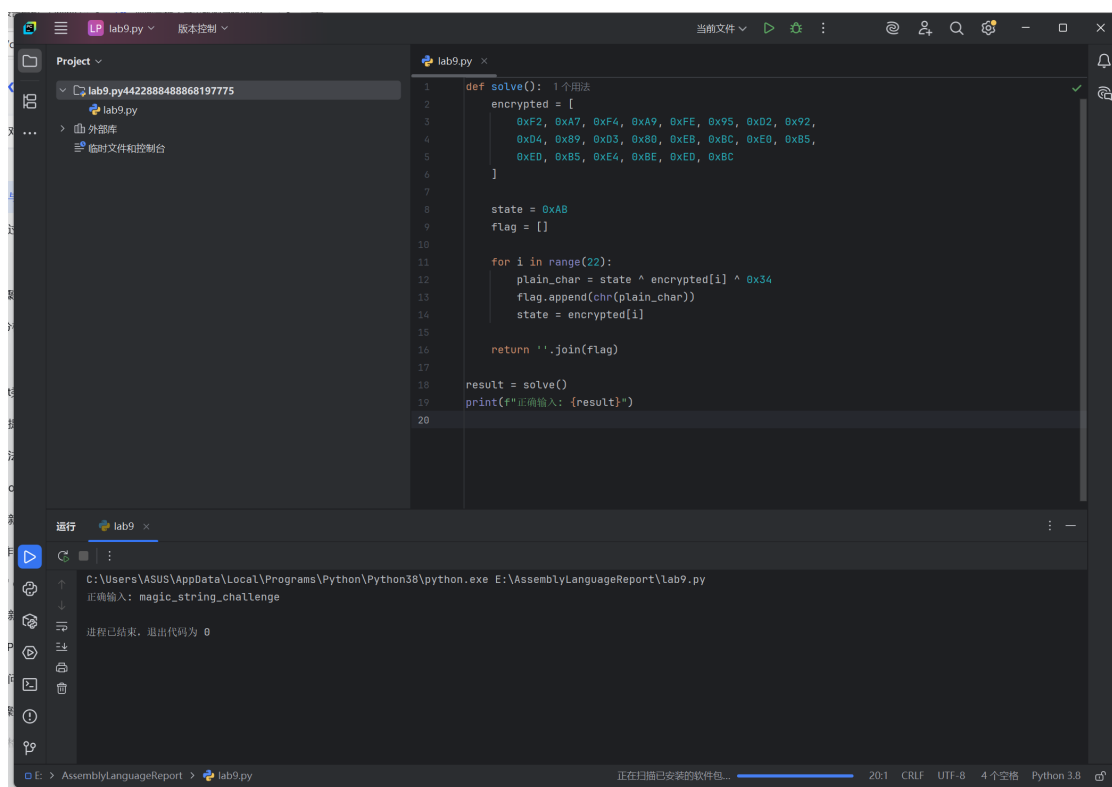


图 6.8: task4 python 代码运行截图

结果为: magic_string_challenge

6.4 数据结构

核心数据结构是字符数组，搭配指针实现灵活访问。

7 实验结果

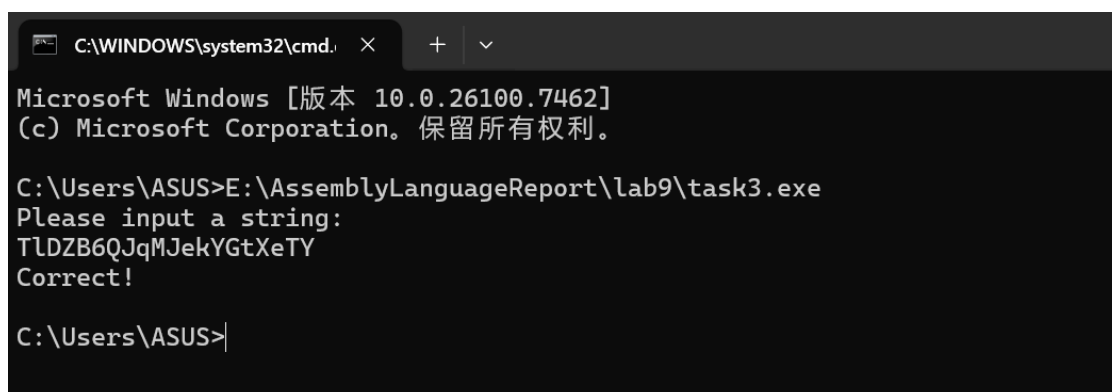
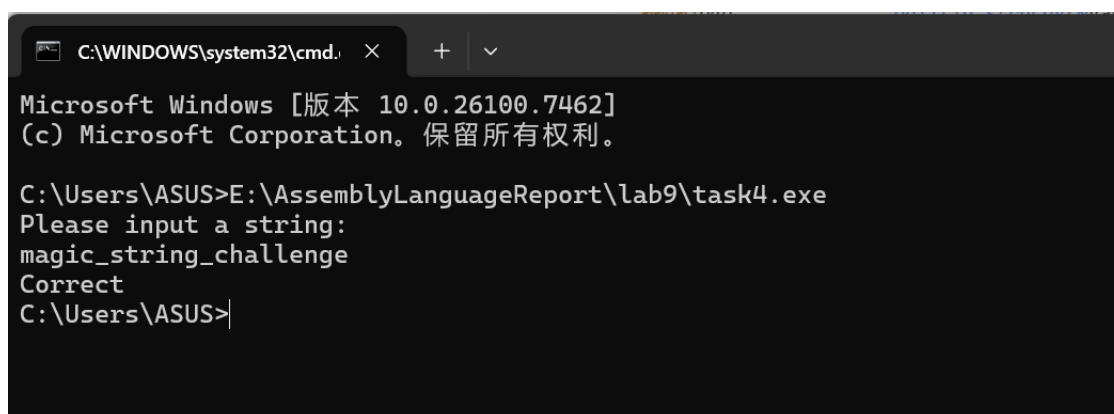


图 7.9: task3 实验结果截图

得到了 “Correct!”



```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows [版本 10.0.26100.7462]
(c) Microsoft Corporation。保留所有权利。

C:\Users\ASUS>E:\AssemblyLanguageReport\lab9\task4.exe
Please input a string:
magic_string_challenge
Correct
C:\Users\ASUS>
```

图 7.10: task4 实验结果截图

得到了”Correct”